SEKOLAH TINGGI TEKNOLOGI TERPADU NURUL FIKRI

LAPORAN UJIAN TENGAH SEMESTER (UTS)

AUDIT KEAMANAN APLIKASI WEB MENGGUNAKAN STATIC APPLICATION SECURITY TESTING (SAST) DENGAN SONARQUBE

Studi Kasus: STT-NF Blog Platform (PHP/Laravel)



Disusun oleh:

KELOMPOK 6

No	Nama Anggota	NIM
1	Firmansyah Dzakwan Arifien	0110223282
2	Azzam Fikri	0110225110
3	Dhiya Nailah Rizqya	0110225095
4	Ghifana Adzkia Ahmad	0110225044
5	Rizky Putri	0110225064
6	Muhammad Fathir Lubis	0110225005
7	Muhammad Bintang Siregar	0110225058

Mata Kuliah : DevSecOps
Dosen Pengampu : April Rustianto, S.Komp., M.T., CCIE
Tanggal UTS : ..... / ..... / 2026

PROGRAM STUDI TEKNIK INFORMATIKA
DEPOK
2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Kelompok 6 berperan sebagai tim DevSecOps yang baru bergabung di sebuah perusahaan startup. Tugas pertama tim adalah melakukan audit keamanan terhadap aplikasi web yang sudah berjalan, yaitu **STT-NF Blog Platform**, sebuah aplikasi manajemen blog berbasis PHP/Laravel (source code `kelompok-A-blog-platform` dari repositori dosen). Audit dilakukan dengan pendekatan Static Application Security Testing (SAST) memakai SonarQube, kemudian kerentanan yang ditemukan diperbaiki dan diverifikasi ulang dengan scan kedua.

Aplikasi ini terdiri dari empat controller (Article, Auth, File, Admin), dua model Eloquent (Post, User), middleware CSRF, konfigurasi database, dan file route. Ukurannya kecil (140 baris kode), sehingga seluruh hasil temuan SonarQube dapat dicek ulang satu per satu secara manual oleh tim.

1.2 Static Application Security Testing (SAST)

SAST adalah metode pengujian keamanan yang menganalisis source code tanpa menjalankan aplikasinya (white-box testing). Tool SAST mem-parsing kode menjadi struktur seperti Abstract Syntax Tree, lalu mencocokkannya dengan aturan (rule) yang mewakili pola kode tidak aman, misalnya fungsi hash yang lemah, kredensial yang ditulis langsung di kode, atau proteksi framework yang dimatikan.

Kelebihan utama SAST adalah bisa dijalankan sejak tahap development (prinsip **shift left**), cepat, dan mudah diotomasi di pipeline CI/CD, sehingga kerentanan ditemukan sebelum aplikasi dirilis dan biaya perbaikannya jauh lebih murah. Kekurangannya, SAST tidak melihat perilaku aplikasi saat runtime dan hasilnya sangat bergantung pada kelengkapan rule yang dipakai. Karena itu SAST biasanya dipadukan dengan DAST dan review kode manual.

1.3 SonarQube

SonarQube adalah platform continuous inspection buatan SonarSource untuk mengukur kualitas dan keamanan kode di lebih dari 30 bahasa, termasuk PHP. Kode dianalisis oleh **SonarScanner** di sisi klien, lalu hasilnya dikirim ke server SonarQube dan ditampilkan di dashboard per project.

Versi yang dipakai pada audit ini adalah **SonarQube Community Build v26.9** dalam mode MQR (Multi-Quality Rule). Pada mode ini setiap issue dikelompokkan berdasarkan **software quality** yang terdampak, yaitu Security, Reliability, dan Maintainability, dengan lima tingkat severity: **Blocker, High, Medium, Low, dan Info**. Setiap rule keamanan juga dipetakan ke standar OWASP Top 10 dan CWE. Untuk menentukan lolos tidaknya kode, SonarQube memakai **Quality Gate**, yaitu kumpulan syarat (misalnya tidak boleh ada issue baru) yang dievaluasi pada setiap analisis.

1.4 Tujuan dan Ruang Lingkup

1. Menjalankan SonarQube dan melakukan scanning terhadap project PHP/Laravel STT-NF Blog Platform.
2. Menganalisis setiap temuan SonarQube: severity, lokasi file dan baris, kategori OWASP, serta dampaknya.
3. Memperbaiki kerentanan yang ditemukan (minimal dua; tim memperbaiki seluruh temuan SonarQube).
4. Memverifikasi perbaikan dengan re-scan pada project SonarQube terpisah dan membandingkan hasilnya.

Ruang lingkup scan mencakup folder `app/`, `routes/`, dan `config/`. Folder `config/` ditambahkan ke `sonar.sources` (aslinya hanya `app,routes`) karena berisi konfigurasi database yang relevan untuk audit keamanan.

BAB II HASIL SCANNING

2.1 Lingkungan dan Langkah Scanning

Tabel 2.1 *Lingkungan pengujian SAST*

Server SonarQube	SonarQube Community Build v26.9.0.129388 (self-hosted), https://sonarqube.cyphera.my.id
Scanner	SonarScanner CLI 7.2.0.5079 (Windows x64)
Analyzer	SonarPHP 3.60, Text & Secrets 2.49
Quality Profile / Gate	Sonar way (PHP: 176 rule aktif, Secrets: 30 rule aktif) / Sonar way
Project sebelum perbaikan	uts-kelompok6-blog-sebelum (versi 1.0)
Project sesudah perbaikan	uts-kelompok6-blog-sesudah (versi 1.1)
Tanggal scan	Selasa, 27 Oktober 2026

Scan sebelum dan sesudah perbaikan sengaja dijalankan ke **dua project SonarQube yang berbeda**, supaya hasil scan awal tetap utuh sebagai bukti (tidak tertimpa oleh scan ulang) dan kedua kondisi bisa dibandingkan berdampingan. Langkah yang dilakukan:

1. Membuat dua project baru di SonarQube dan sebuah analysis token. Token tidak ditulis di file, melainkan diberikan saat scanner dijalankan.
2. Menghapus `sonar.login` dan `sonar.password` dari `sonar-project.properties`. Kedua properti ini sudah tidak didukung SonarQube versi terbaru (scanner berhenti dengan error) dan menyimpan kredensial di repositori juga merupakan praktik yang tidak aman.
3. Menjalankan SonarScanner pada source code asli ke project sebelum perbaikan.
4. Menganalisis temuan, memperbaiki kode, lalu menjalankan SonarScanner pada kode hasil perbaikan ke project sesudah perbaikan.

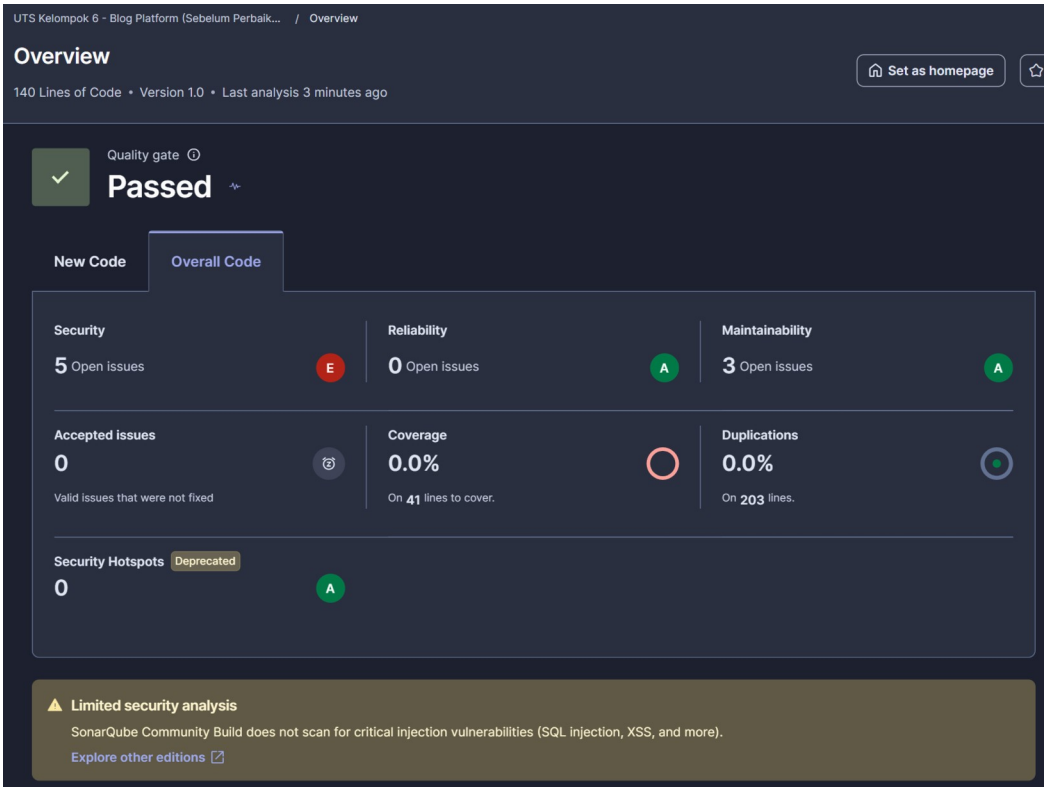
Perintah scan yang digunakan (token disamarkan):

```
sonar-scanner \
  -Dsonar.host.url=https://sonarqube.cyphera.my.id \
  -Dsonar.token=<ANALYSIS_TOKEN> \
  -Dsonar.projectKey=uts-kelompok6-blog-sebelum \
  -Dsonar.sources=app,routes,config \
  -Dsonar.projectVersion=1.0
```

2.2 Dashboard SonarQube

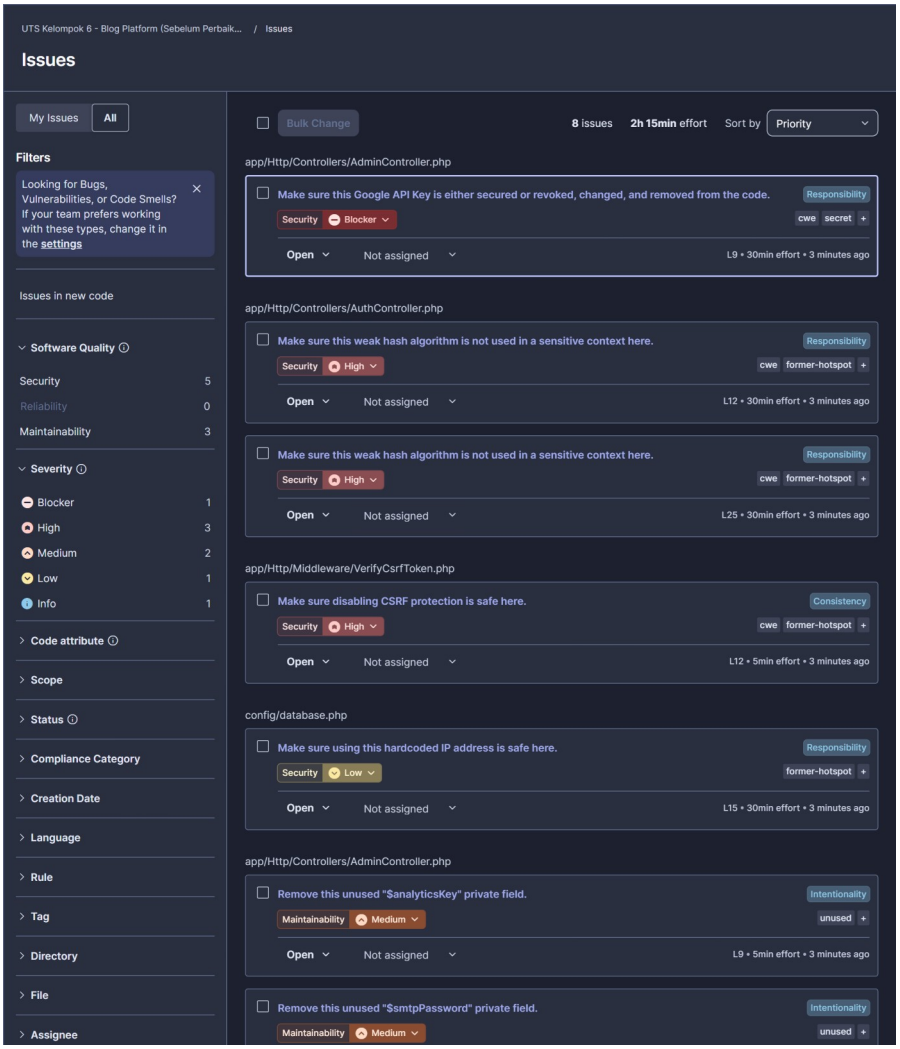
Gambar 2.1 menunjukkan dashboard project sebelum perbaikan. SonarQube menemukan **5 issue Security** dengan Security Rating **E** (rating terburuk, karena ada

issue Blocker), **0 issue Reliability**, dan **3 issue Maintainability**. Quality Gate berstatus Passed karena ini analisis pertama: Quality Gate "Sonar way" hanya menilai kode baru (new code), sedangkan pada analisis pertama seluruh kode dianggap kode lama. Artinya status Passed di sini tidak berarti kode aman.



Gambar 2.1 *Dashboard SonarQube project sebelum perbaikan (Overall Code)*

Pada bagian bawah dashboard SonarQube juga menampilkan peringatan **Limited security analysis**: edisi Community Build tidak menjalankan taint analysis untuk kerentanan injeksi (SQL Injection, XSS, dan sejenisnya). Hal ini penting untuk analisis di Bab III.



Gambar 2.2 *Daftar issue project sebelum perbaikan beserta filter Software Quality dan Severity*

2.3 Ringkasan Temuan

Tabel 2.2 merangkum jumlah temuan per kategori (software quality) dan per severity sesuai tampilan SonarQube, sedangkan Tabel 2.3 memuat padanan dalam klasifikasi lama (tipe issue) yang masih dipakai di banyak dokumentasi.

Tabel 2.2 *Jumlah temuan per kategori (software quality) per severity*

Kategori	Blocker	High	Medium	Low	Info	Total
Security	1	3	0	1	0	5
Reliability	0	0	0	0	0	0
Maintainability	0	0	2	0	1	3
Total	1	3	2	1	1	8

Tabel 2.3 *Jumlah temuan per tipe issue (klasifikasi lama) per severity*

Tipe Issue	Blocker	Critical	Major	Minor	Info	Total
Vulnerability	1	3	0	0	0	4
Bug	0	0	0	0	0	0
Code Smell	0	0	2	1	1	4
Security Hotspot	-	-	-	-	-	0

Catatan: hardcoded IP address (`php:S1313`) bertipe Code Smell pada klasifikasi lama, tetapi pada mode MQR dihitung sebagai issue Security severity Low. Itulah sebabnya Security berjumlah 5 sementara Vulnerability berjumlah 4. Security Hotspot bernilai 0 karena pada versi ini rule hotspot (MD5, CSRF, IP) sudah dilaporkan langsung sebagai issue dengan tag `former-hotspot`.

Tabel 2.4 *Daftar lengkap temuan SonarQube*

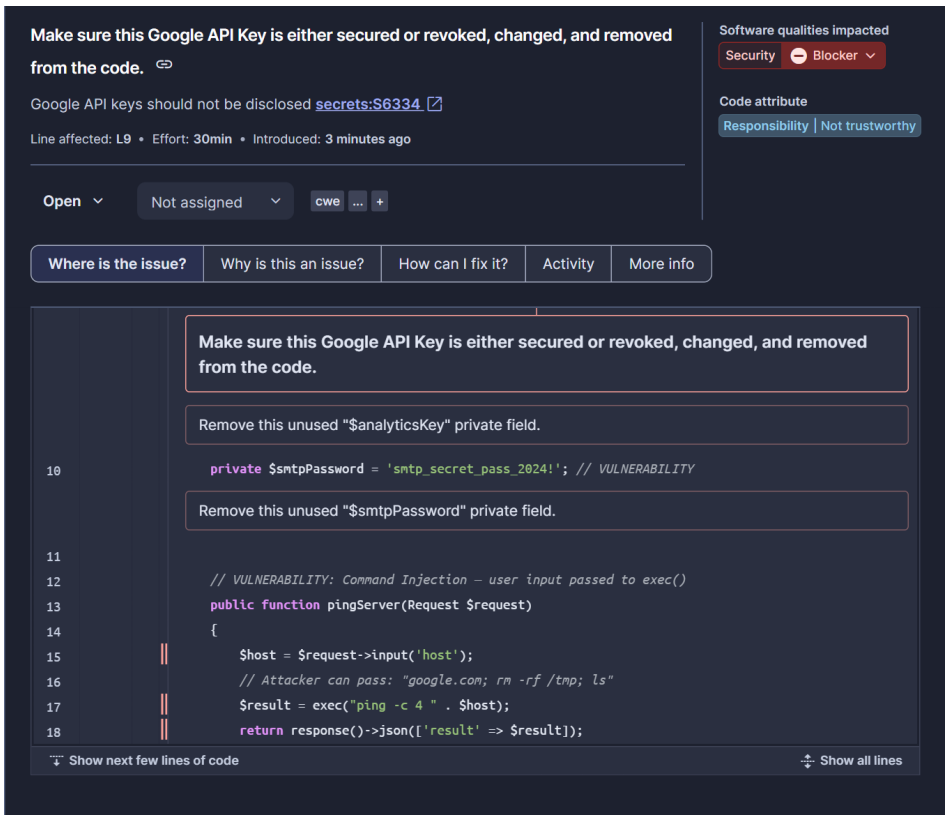
No	Rule	Pesan SonarQube	File : Baris	Severity
1	secrets:S6334	Make sure this Google API Key is either secured or revoked, changed, and removed from the code.	AdminController.php : 9	Blocker
2	php:S4790	Make sure this weak hash algorithm is not used in a sensitive context here.	AuthController.php : 12	High
3	php:S4790	Make sure this weak hash algorithm is not used in a sensitive context here.	AuthController.php : 25	High
4	php:S4502	Make sure disabling CSRF protection is safe here.	VerifyCsrfToken.php : 12	High
5	php:S1313	Make sure using this hardcoded IP address is safe here.	config/ database.php : 15	Low
6	php:S1068	Remove this unused "$analyticsKey" private field.	AdminController.php : 9	Medium
7	php:S1068	Remove this unused "$smtpPassword" private field.	AdminController.php : 10	Medium
8	php:S1135	Complete the task associated to this "TODO" comment.	VerifyCsrfToken.php : 10	Info

BAB III ANALISIS HASIL KERENTANAN

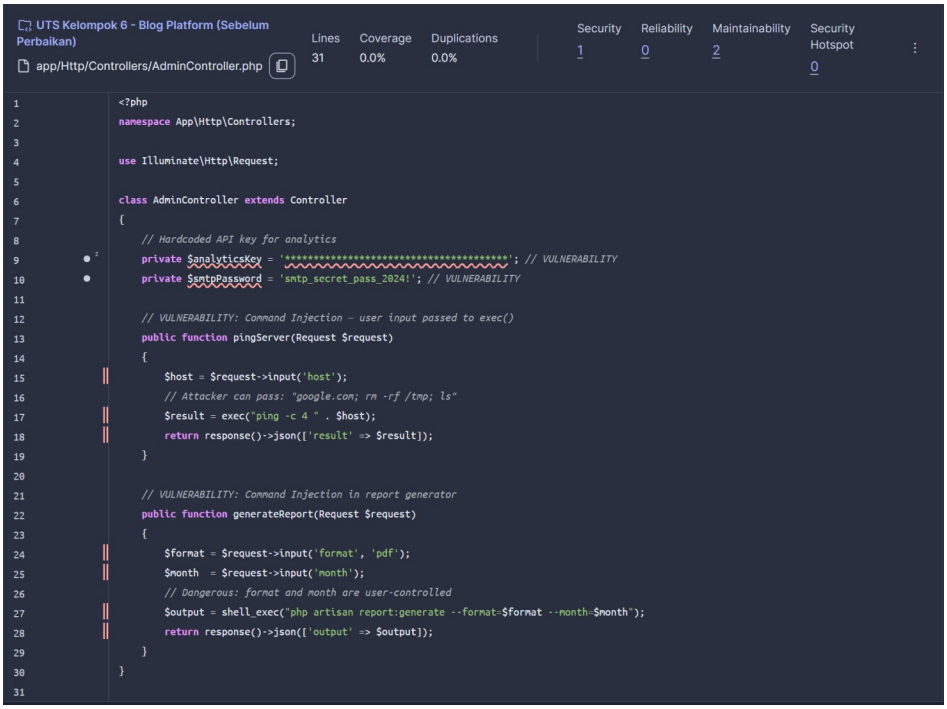
Bab ini membahas setiap temuan keamanan dari SonarQube (subbab 3.1 sampai 3.4), temuan maintainability (3.5), lalu temuan tambahan dari review kode manual yang tidak terdeteksi oleh SonarQube Community Build (3.6). Screenshot kode bermasalah diambil dari halaman issue dan code viewer SonarQube; baris yang ditandai garis bergelombang merah adalah lokasi issue.

3.1 Hardcoded Secret: Google API Key dan Password SMTP

Rule	secrets:S6334, Google API keys should not be disclosed
Severity	Blocker (Security). Klasifikasi lama: Vulnerability, Blocker
File & baris	app/Http/Controllers/AdminController.php, baris 9 (API key) dan baris 10 (password SMTP)
OWASP	A07:2021 Identification and Authentication Failures (OWASP 2017: A3 Sensitive Data Exposure)
CWE	CWE-798 Use of Hard-coded Credentials, CWE-259 Use of Hard-coded Password



Gambar 3.1 *Issue secrets:S6334 di SonarQube (nilai key otomatis disamarkan oleh SonarQube)*



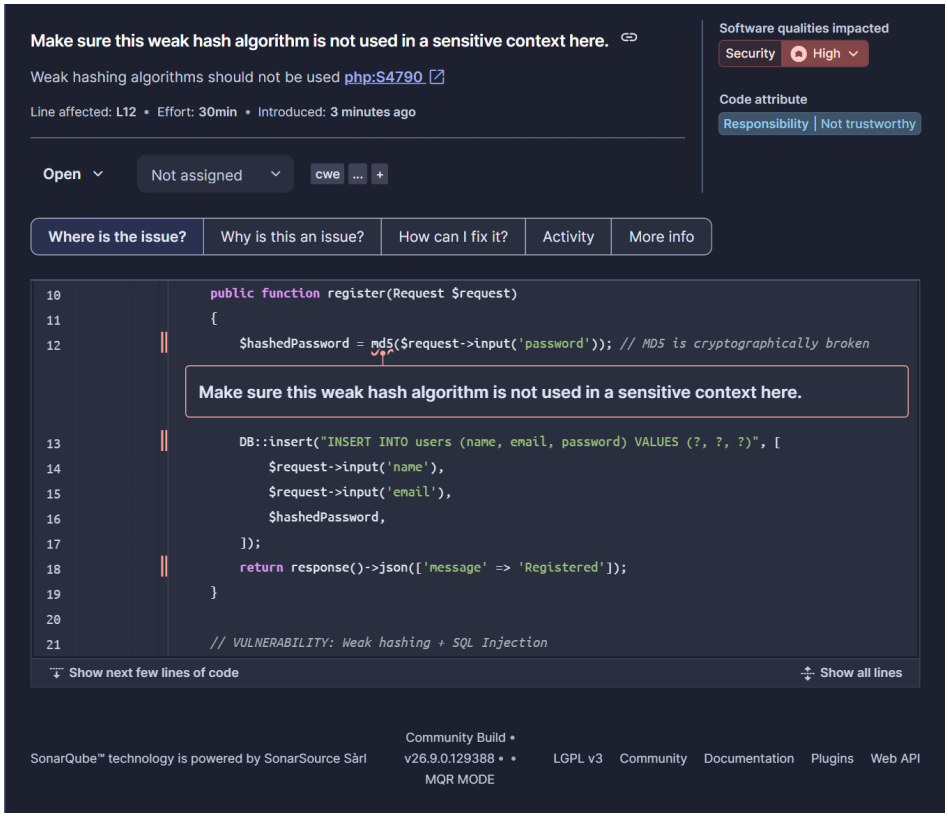
Gambar 3.2 *Kode AdminController.php sebelum perbaikan (baris 9-10 hardcoded secret, baris 17 dan 27 command injection)*

Mengapa berbahaya:

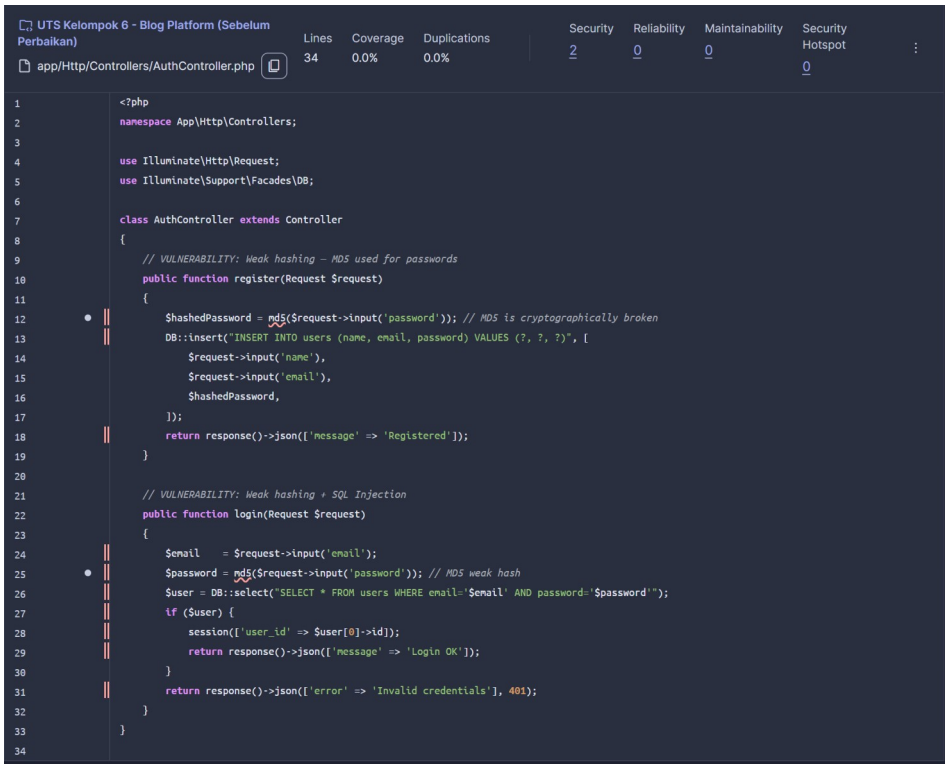
- API key Google dan password SMTP ditulis sebagai string literal di dalam class. Siapa pun yang punya akses ke repositori, termasuk riwayat git, salinan zip, atau kebocoran repositori, langsung mendapatkan secret tersebut.
- API key yang bocor dapat dipakai pihak lain untuk memanggil layanan Google atas nama perusahaan sehingga menimbulkan tagihan atau penyalahgunaan kuota. Password SMTP dapat dipakai untuk mengirim email phishing dari domain resmi perusahaan.
- Secret yang sudah masuk riwayat git tidak hilang hanya dengan menghapus barisnya; secret itu harus dianggap bocor dan dirotasi (revoke lalu buat baru).

3.2 Weak Hashing: Password Disimpan dengan MD5

Rule	php:S4790, Weak hashing algorithms should not be used
Severity	High (Security). Klasifikasi lama: Vulnerability, Critical
File & baris	app/Http/Controllers/AuthController.php, baris 12 (register) dan baris 25 (login)
OWASP	A02:2021 Cryptographic Failures (OWASP 2017: A3 dan A6)
CWE	CWE-1240 Use of a Cryptographic Primitive with a Risky Implementation (juga terkait CWE-328 dan CWE-916)



Gambar 3.3 *Issue php:S4790 pada fungsi register (baris 12)*



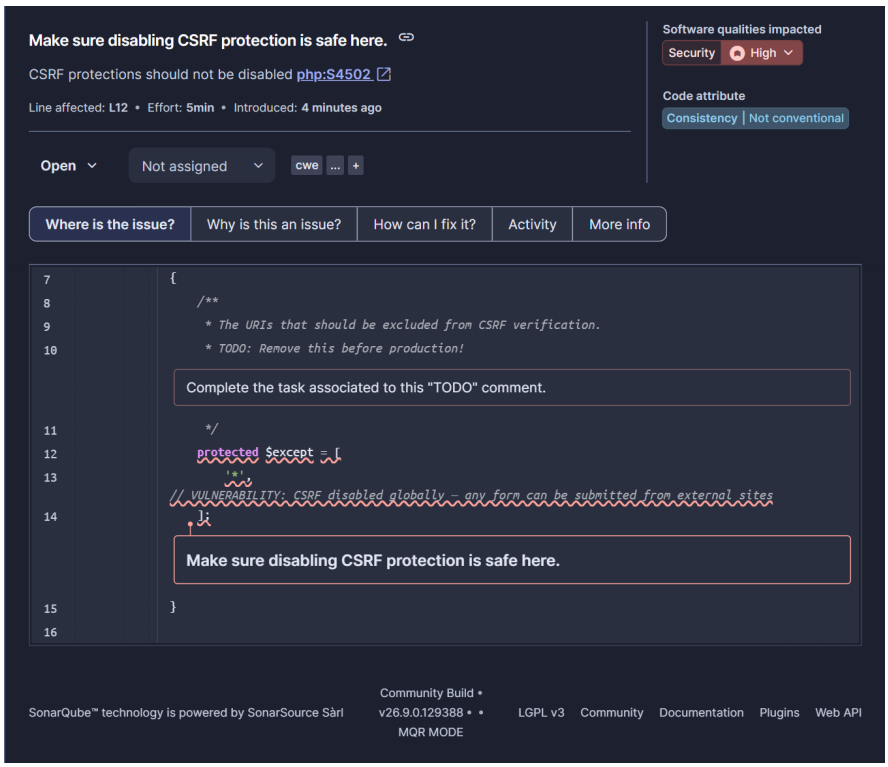
Gambar 3.4 *Kode AuthController.php sebelum perbaikan (md5 di baris 12 dan 25, query login baris 26)*

Mengapa berbahaya:

- MD5 adalah fungsi hash cepat tanpa salt. GPU modern dapat menghitung miliaran hash MD5 per detik, sehingga password pengguna yang bocor dari database bisa dipecahkan dengan brute force atau rainbow table dalam hitungan menit.
- Karena tidak ada salt, dua pengguna dengan password yang sama akan memiliki hash yang sama, sehingga satu password yang berhasil dipecahkan langsung membuka akun lain.
- Pada fungsi login, hasil MD5 dan email juga digabung langsung ke query SQL (baris 26). Ini SQL Injection yang tidak dilaporkan SonarQube (lihat 3.6), sehingga penyerang bisa login tanpa password, misalnya dengan email `admin@x.com' --`.

3.3 Proteksi CSRF Dinonaktifkan untuk Semua Route

Rule	php:S4502, CSRF protections should not be disabled
Severity	High (Security). Klasifikasi lama: Vulnerability, Critical
File & baris	app/Http/Middleware/VerifyCsrfToken.php, baris 12-13 (`$except = ['*']`)
OWASP	A01:2021 Broken Access Control (OWASP 2017: A6 Security Misconfiguration)
CWE	CWE-352 Cross-Site Request Forgery (CSRF)



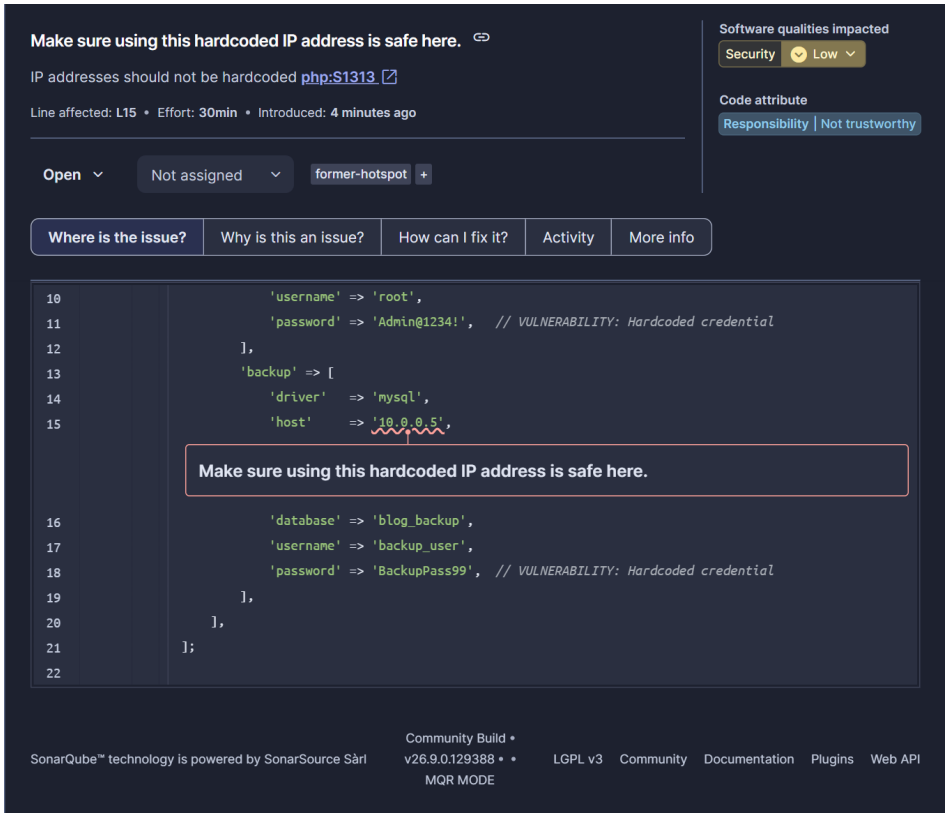
Gambar 3.5 *Issue php:S4502 beserta issue TODO (php:S1135) pada VerifyCsrfToken.php*

Mengapa berbahaya:

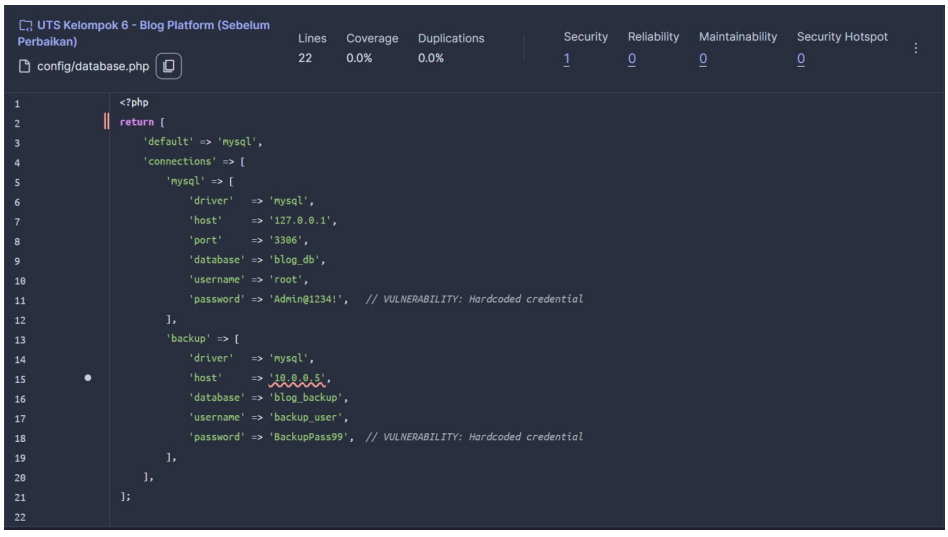
- Wildcard `*` pada `$except` membuat middleware CSRF Laravel melewati semua route, sehingga token CSRF tidak pernah diperiksa.
- Penyerang cukup membuat halaman web berisi form tersembunyi yang otomatis dikirim ke aplikasi. Jika korban yang sedang login membuka halaman itu, browser ikut mengirim cookie sesi dan aksi berjalan atas nama korban, misalnya menghapus post, membuat artikel, atau memanggil endpoint admin.
- Komentar `TODO: Remove this before production!` menunjukkan konfigurasi ini sebenarnya hanya untuk pengembangan, tetapi ikut terbawa ke kode yang berjalan.

3.4 Hardcoded IP Address dan Kredensial Database

Rule	php:S1313, IP addresses should not be hardcoded
Severity	Low (Security). Klasifikasi lama: Code Smell, Minor
File & baris	config/database.php, baris 15 (IP 10.0.0.5); terkait password di baris 11 dan 18
OWASP	A01:2021 Broken Access Control (OWASP 2017: A3 Sensitive Data Exposure)
CWE	CWE-547 Use of Hard-coded, Security-relevant Constants



Gambar 3.6 *Issue php:S1313 pada config/database.php*



Gambar 3.7 *Kode config/database.php sebelum perbaikan*

Mengapa berbahaya:

- Alamat IP server backup internal (10.0.0.5) memberi informasi topologi jaringan kepada siapa pun yang membaca kode, dan menyulitkan perpindahan server karena alamat harus diubah di kode.
- Pada file yang sama terdapat password database root `Admin@1234!` dan password backup `BackupPass99` dalam bentuk teks biasa. SonarQube tidak menandai keduanya (rule hard-coded password tidak memicu pada array konfigurasi ini),

tetapi dampaknya lebih besar: kebocoran source code langsung memberi akses penuh ke database produksi dan backup.

3.5 Temuan Maintainability

Tabel 3.1 *Temuan maintainability dan kaitannya dengan keamanan*

Rule	Lokasi	Severity	Keterangan
php:S1068 (x2)	AdminController.php : 9, 10	Medium	Field `$analyticsKey` dan `$smtpPassword` tidak pernah dipakai. Secret yang tidak dipakai tetap bocor bila ditulis di kode, jadi field ini sebaiknya dihapus.
php:S1135	VerifyCsrfToken.php : 10	Info	Komentar TODO yang belum diselesaikan, dan isinya justru peringatan bahwa CSRF masih dimatikan.

3.6 Temuan Review Manual (Tidak Terdeteksi SonarQube Community Build)

SonarQube Community Build hanya memakai analisis berbasis pola dan tidak memiliki taint analysis (pelacakan aliran input pengguna ke fungsi berbahaya). Fitur tersebut hanya ada di edisi berbayar, sesuai peringatan "Limited security analysis" pada dashboard. Akibatnya kerentanan injeksi yang justru paling kritis di aplikasi ini tidak muncul sebagai issue. Tim menelusuri setiap controller secara manual dan menemukan kerentanan berikut:

Tabel 3.2 *Temuan review manual*

Kerentanan	Lokasi	OWASP 2021	Dampak
SQL Injection	ArticleController.php : 14, 24; AuthController.php : 26	A03 Injection	Input `q`, `category`, `sort`, dan `email` digabung langsung ke query. Penyerang dapat membaca seluruh tabel atau melewati login.
OS Command Injection	AdminController.php : 17, 27	A03 Injection	Input `host`, `format`, dan `month` dikirim ke `exec()` / `shell_exec()`. Contoh `host=google.com; cat .env` menjalankan perintah apa pun di server.
Path Traversal	FileController.php : 13, 21	A01 Broken Access Control	Parameter `file` / `path` tanpa validasi, sehingga `../../.env` dapat membaca file rahasia aplikasi.

Kerentanan	Lokasi	OWASP 2021	Dampak
IDOR	ArticleController.php : 33	A01 Broken Access Control	Setiap pengguna dapat menghapus post milik orang lain hanya dengan mengganti ID (sekaligus SQL Injection melalui `$id`).
Mass Assignment	ArticleController.php : 40; Post.php : 8; User.php : 8	A08 Software and Data Integrity Failures	`Post::create($request->all())` dengan `$guarded = []` memungkinkan pengguna mengisi kolom seperti `user_id` atau `is_admin`.
Hardcoded password DB	config/database.php : 11, 18	A07 Identification and Authentication Failures	Password root dan backup database tertulis di source code.

Temuan ini menunjukkan bahwa hasil SonarQube yang "hanya" 4 vulnerability tidak menggambarkan risiko sebenarnya. Hasil SAST tetap perlu dilengkapi review manual, taint analysis, atau DAST.

BAB IV REMEDIASI

Tugas mensyaratkan minimal dua kerentanan diperbaiki. Tim memperbaiki **seluruh 8 temuan SonarQube** (4.1 sampai 4.4) sekaligus kerentanan hasil review manual (4.5). Setiap perbaikan dicatat sebagai commit terpisah di branch `uts/sast-sonarqube-kelompok6`, sehingga diff di bawah ini adalah keluaran `git diff` yang sebenarnya.

4.1 Perbaikan Hardcoded Secret (secrets:S6334, php:S1068)

Kode sebelum vs sesudah (diff). Baris merah bertanda - adalah kode sebelum perbaikan, baris hijau bertanda + adalah kode sesudah perbaikan.

`app/Http/Controllers/AdminController.php`

```
@@ -4,2 +4,4 @@
 use Illuminate\Http\Request;
+use Illuminate\Support\Facades\Artisan;
+use Symfony\Component\Process\Process;

@@ -7,23 +9,34 @@
 {
-    // Hardcoded API key for analytics
-    private $analyticsKey = 'AIzaSyD-9tSr**************************'; // VULNERABILITY
-    private $smtpPassword = 'smtp_secret_pass_2024!'; // VULNERABILITY
+    // FIX: API key analytics & password SMTP tidak lagi ditulis di source code.
+    // Nilainya disimpan di .env dan dibaca lewat config('services.analytics.key')
+    // serta config('mail.mailers.smtp.password') saat dibutuhkan.

-    // VULNERABILITY: Command Injection — user input passed to exec()
+    // FIX: Command Injection, host divalidasi, perintah dijalankan sebagai array argumen
(tanpa shell)
     public function pingServer(Request $request)
     {
-        $host = $request->input('host');
-        // Attacker can pass: "google.com; rm -rf /tmp; ls"
-        $result = exec("ping -c 4 " . $host);
-        return response()->json(['result' => $result]);
+        $validated = $request->validate([
+            'host' => ['required', 'string', 'max:253', 'regex:/^[A-Za-z0-9.-]+$/'],
+        ]);
+
+        $process = new Process(['ping', '-c', '4', $validated['host']]);
+        $process->setTimeout(15);
+        $process->run();
+
+        return response()->json(['result' => $process->getOutput()]);
     }

-    // VULNERABILITY: Command Injection in report generator
+    // FIX: Command Injection, parameter di-whitelist, artisan dipanggil lewat API (tanpa
shell)
     public function generateReport(Request $request)
     {
-        $format = $request->input('format', 'pdf');
-        $month  = $request->input('month');
-        // Dangerous: format and month are user-controlled
-        $output = shell_exec("php artisan report:generate --format=$format --month=$month");
-        return response()->json(['output' => $output]);
+        $validated = $request->validate([
+            'format' => ['sometimes', 'in:pdf,csv,xlsx'],
+            'month'  => ['required', 'date_format:Y-m'],
+        ]);
+
+        Artisan::call('report:generate', [
+            '--format' => $validated['format'] ?? 'pdf',
+            '--month'  => $validated['month'],
+        ]);
+
+        return response()->json(['output' => Artisan::output()]);
```

```
    }
```

Penjelasan perubahan:

- Field `$analyticsKey` dan `$smtpPassword` dihapus dari class. Nilainya dipindahkan ke file `.env` (tidak ikut di-commit) dan dibaca lewat `config/services.php` dengan `env('ANALYTICS_API_KEY')`. File `.env.example` disediakan sebagai daftar variabel yang wajib diisi tanpa nilai rahasia.
- Karena field yang tidak dipakai ikut dihapus, issue `php:S1068` (2 issue) juga selesai.
- Pada fungsi yang sama, command injection diperbaiki: input divalidasi (regex hostname, whitelist format, format bulan `Y-m`), dan perintah dijalankan dengan `Symfony\Process` berupa array argumen serta `Artisan::call()`, sehingga tidak ada lagi shell yang bisa disisipi `;` atau `&&`.
- Langkah operasional di luar kode: API key dan password SMTP yang lama wajib di-revoke dan diganti karena sudah tersimpan di riwayat repositori.



Gambar 4.1 *AdminController.php di project sesudah perbaikan: Security 0, Maintainability 0*

4.2 Perbaikan Weak Hashing MD5 (php:S4790)

Kode sebelum vs sesudah (diff). Baris merah bertanda - adalah kode sebelum perbaikan, baris hijau bertanda + adalah kode sesudah perbaikan.

`app/Http/Controllers/AuthController.php`

```
@@ -5,2 +5,3 @@
 use Illuminate\Support\Facades\DB;
+use Illuminate\Support\Facades\Hash;


@@ -8,10 +9,15 @@
 {
-    // VULNERABILITY: Weak hashing — MD5 used for passwords
+    // FIX: password di-hash dengan bcrypt (Hash::make), bukan MD5
     public function register(Request $request)
     {
-        $hashedPassword = md5($request->input('password')); // MD5 is cryptographically
broken
+        $validated = $request->validate([
+            'name'     => ['required', 'string', 'max:100'],
+            'email'    => ['required', 'email', 'unique:users,email'],
+            'password' => ['required', 'string', 'min:8'],
+        ]);
+
         DB::insert("INSERT INTO users (name, email, password) VALUES (?, ?, ?)", [
-            $request->input('name'),
-            $request->input('email'),
-            $hashedPassword,
+            $validated['name'],
+            $validated['email'],
+            Hash::make($validated['password']),
         ]);
@@ -20,10 +26,14 @@


-    // VULNERABILITY: Weak hashing + SQL Injection
+    // FIX: query memakai parameter binding + verifikasi password dengan Hash::check
     public function login(Request $request)
     {
-        $email    = $request->input('email');
-        $password = md5($request->input('password')); // MD5 weak hash
-        $user = DB::select("SELECT * FROM users WHERE email='$email' AND
password='$password'");
-        if ($user) {
-            session(['user_id' => $user[0]->id]);
+        $credentials = $request->validate([
+            'email'    => ['required', 'email'],
+            'password' => ['required', 'string'],
+        ]);
+
+        $user = DB::selectOne("SELECT * FROM users WHERE email = ?",
[$credentials['email']]);
+        if ($user && Hash::check($credentials['password'], $user->password)) {
+            $request->session()->regenerate();
+            session(['user_id' => $user->id]);
             return response()->json(['message' => 'Login OK']);
```

Penjelasan perubahan:

- `md5()` diganti `Hash::make()` milik Laravel yang secara default memakai **bcrypt**: lambat secara sengaja (cost factor), memiliki salt acak per password, dan cost-nya bisa dinaikkan seiring waktu.
- Proses login tidak lagi membandingkan hash di dalam query. User dicari berdasarkan email dengan **parameter binding** (`?`), lalu password diverifikasi dengan `Hash::check()`. Perubahan ini sekaligus menutup SQL Injection di baris 26.

- Ditambahkan validasi input (`email`, panjang password minimal 8) dan `session()->regenerate()` setelah login berhasil untuk mencegah session fixation.
- Catatan migrasi: hash MD5 lama tidak bisa dikonversi. Pengguna lama perlu reset password, atau hash di-upgrade saat pengguna login berikutnya.



Gambar 4.2 *AuthController.php di project sesudah perbaikan: Security 0*

4.3 Perbaikan CSRF Dinonaktifkan (php:S4502, php:S1135)

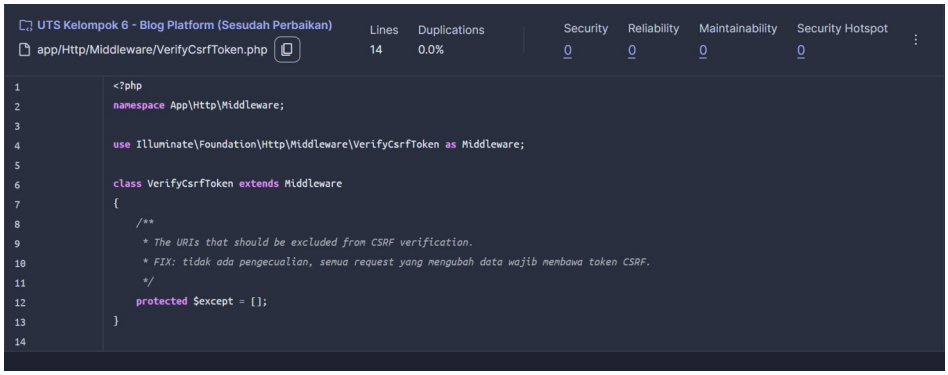
Kode sebelum vs sesudah (diff). Baris merah bertanda - adalah kode sebelum perbaikan, baris hijau bertanda + adalah kode sesudah perbaikan.

`app/Http/Middleware/VerifyCsrfToken.php`

```
@@ -9,7 +9,5 @@
      * The URIs that should be excluded from CSRF verification.
-     * TODO: Remove this before production!
+     * FIX: tidak ada pengecualian, semua request yang mengubah data wajib membawa token CSRF.
      */
-    protected $except = [
-        '*',  // VULNERABILITY: CSRF disabled globally — any form can be submitted from external sites
-    ];
+    protected $except = [];
 }
```

Penjelasan perubahan:

- Wildcard `*` dihapus sehingga `$except` menjadi array kosong. Seluruh request POST, PUT, PATCH, dan DELETE kembali wajib membawa token CSRF yang valid.
- Komentar TODO yang menjadi issue `php:S1135` diganti keterangan bahwa tidak ada pengecualian.
- Jika nanti ada endpoint API stateless yang memang tidak memakai sesi, endpoint itu sebaiknya ditempatkan di `routes/api.php` dengan autentikasi token, bukan dengan mematikan CSRF secara global.



Gambar 4.3 *VerifyCsrfToken.php di project sesudah perbaikan*

4.4 Perbaikan Hardcoded IP dan Kredensial Database (php:S1313)

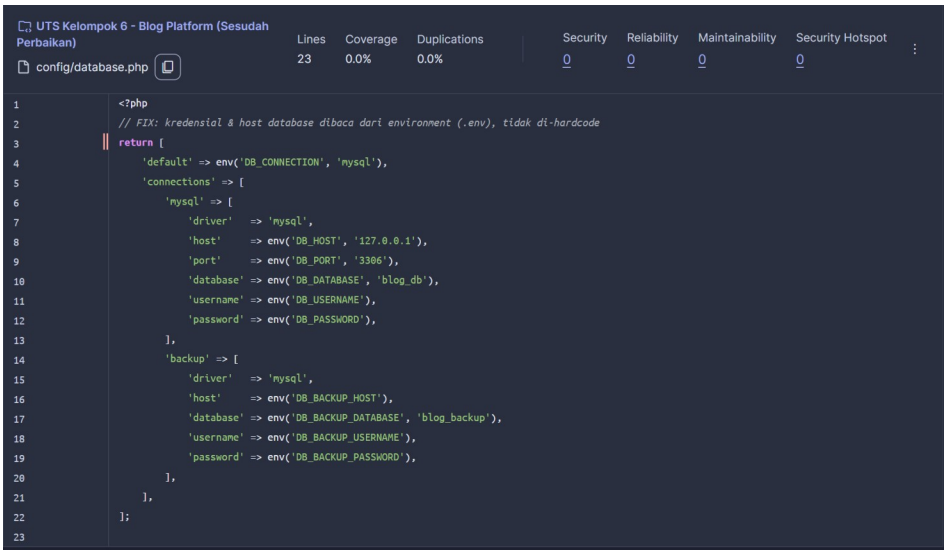
Kode sebelum vs sesudah (diff). Baris merah bertanda - adalah kode sebelum perbaikan, baris hijau bertanda + adalah kode sesudah perbaikan.

`config/database.php`

```
@@ -1,4 +1,5 @@
 <?php
+// FIX: kredensial & host database dibaca dari environment (.env), tidak di-hardcode
 return [
-    'default' => 'mysql',
+    'default' => env('DB_CONNECTION', 'mysql'),
     'connections' => [
@@ -6,7 +7,7 @@
             'driver'   => 'mysql',
-            'host'     => '127.0.0.1',
-            'port'     => '3306',
-            'database' => 'blog_db',
-            'username' => 'root',
-            'password' => 'Admin@1234!',   // VULNERABILITY: Hardcoded credential
+            'host'     => env('DB_HOST', '127.0.0.1'),
+            'port'     => env('DB_PORT', '3306'),
+            'database' => env('DB_DATABASE', 'blog_db'),
+            'username' => env('DB_USERNAME'),
+            'password' => env('DB_PASSWORD'),
         ],
@@ -14,6 +15,6 @@
             'driver'   => 'mysql',
-            'host'     => '10.0.0.5',
-            'database' => 'blog_backup',
-            'username' => 'backup_user',
-            'password' => 'BackupPass99',  // VULNERABILITY: Hardcoded credential
+            'host'     => env('DB_BACKUP_HOST'),
+            'database' => env('DB_BACKUP_DATABASE', 'blog_backup'),
+            'username' => env('DB_BACKUP_USERNAME'),
+            'password' => env('DB_BACKUP_PASSWORD'),
         ],
```

Penjelasan perubahan:

- Host, nama database, username, dan password untuk koneksi utama maupun backup dibaca dari environment variable (`DB_HOST`, `DB_PASSWORD`, `DB_BACKUP_HOST`, dan seterusnya), mengikuti pola standar konfigurasi Laravel.
- Password `Admin@1234!` dan `BackupPass99` yang pernah tertulis di kode wajib diganti di server database.



Gambar 4.4 *config/database.php di project sesudah perbaikan*

4.5 Perbaikan Tambahan dari Review Manual

Selain temuan SonarQube, tim memperbaiki kerentanan dari Tabel 3.2. Ringkasan perubahannya ada di Tabel 4.1 dan diff lengkapnya di bawah tabel.

Tabel 4.1 *Ringkasan perbaikan tambahan*

Kerentanan	Perbaikan
SQL Injection (search, filter, login)	Parameter binding pada `DB::select`, query builder Eloquent untuk filter, whitelist kolom `sort` (`created_at`, `title`).
Command Injection	Validasi input dan eksekusi tanpa shell (`Symfony\Process`, `Artisan::call`), lihat 4.1.
Path Traversal	Fungsi `safePath()`: path di-resolve dengan `realpath()` lalu wajib berada di dalam direktori yang diizinkan; preview hanya menerima MIME JPEG/PNG.
IDOR	`Post::findOrFail($id)` lalu cek `user_id` pemilik post sama dengan user di sesi; jika tidak, respons 403.
Mass Assignment	`$fillable` eksplisit pada Post dan User, `$hidden` untuk password, `store()` hanya memakai data hasil validasi dan `user_id` diambil dari sesi.

Kerentanan	Perbaikan
Kredensial di sonar-project.properties	`sonar.login/sonar.password` dihapus; host dan token diberikan lewat environment `SONAR_HOST_URL` dan `SONAR_TOKEN`.

`app/Http/Controllers/ArticleController.php`

```
@@ -3,2 +3,3 @@

+use App\Models\Post;
 use Illuminate\Http\Request;
@@ -8,8 +9,7 @@
 {
-    // VULNERABILITY: SQL Injection via raw query with string concatenation
+    // FIX: SQL Injection, memakai parameter binding
     public function search(Request $request)
     {
-        $keyword = $request->input('q');
-        // Dangerous: user input directly injected into SQL
-        $articles = DB::select("SELECT * FROM posts WHERE title LIKE '%" . $keyword . "%' OR
content LIKE '%" . $keyword . "%'");
+        $keyword = '%' . $request->input('q', '') . '%';
+        $articles = DB::select("SELECT * FROM posts WHERE title LIKE ? OR content LIKE ?",
[$keyword, $keyword]);
         return response()->json($articles);
@@ -17,10 +17,13 @@

-    // VULNERABILITY: SQL Injection in filter
+    // FIX: SQL Injection, category di-binding, kolom sort di-whitelist
     public function filterByCategory(Request $request)
     {
-        $category = $request->input('category');
-        $sort     = $request->input('sort', 'created_at');
-        // Dangerous: both category and sort are user-controlled
-        $query = "SELECT * FROM posts WHERE category = '$category' ORDER BY $sort DESC";
-        $posts = DB::select($query);
+        $validated = $request->validate([
+            'category' => ['required', 'string', 'max:50'],
+            'sort'     => ['sometimes', 'in:created_at,title'],
+        ]);
+
+        $posts = Post::where('category', $validated['category'])
+            ->orderByDesc($validated['sort'] ?? 'created_at')
+            ->get();
         return response()->json($posts);
@@ -28,7 +31,8 @@

-    // VULNERABILITY: IDOR — no ownership check
+    // FIX: IDOR, hanya pemilik post yang boleh menghapus
     public function deletePost(Request $request, $id)
     {
-        // Any authenticated user can delete any post by guessing the ID
-        DB::delete("DELETE FROM posts WHERE id = $id");
+        $post = Post::findOrFail($id);
+        abort_unless((int) $post->user_id === (int) session('user_id'), 403);
+        $post->delete();
         return response()->json(['message' => 'Post deleted']);
@@ -36,6 +40,12 @@

-    // VULNERABILITY: Mass Assignment via create()
+    // FIX: Mass Assignment, hanya field yang divalidasi, user_id diambil dari sesi
     public function store(Request $request)
     {
-        $post = Post::create($request->all()); // passes all input including 'is_admin',
'user_id'
+        $validated = $request->validate([
+            'title'    => ['required', 'string', 'max:255'],
+            'content'  => ['required', 'string'],
+            'category' => ['required', 'string', 'max:50'],
+        ]);
+
+        $post = Post::create($validated + ['user_id' => session('user_id')]);
```

```
         return response()->json($post);
```

`app/Http/Controllers/FileController.php`

```
@@ -7,8 +7,14 @@
 {
-    // VULNERABILITY: Path Traversal — user controls file path
+    // FIX: Path Traversal, path di-resolve dengan realpath() lalu dipastikan tetap di dalam direktori yang diizinkan
+    private function safePath(string $baseDir, ?string $relative): string
+    {
+        $base = realpath($baseDir);
+        $full = realpath($baseDir . DIRECTORY_SEPARATOR . (string) $relative);
+        abort_if($base === false || $full === false || !str_starts_with($full, $base . DIRECTORY_SEPARATOR), 404);
+        return $full;
+    }
+
     public function download(Request $request)
     {
-        $filename = $request->input('file');
-        // Attacker can pass: ../../.env or ../../../etc/passwd
-        $path = storage_path('app/public/' . $filename);
+        $path = $this->safePath(storage_path('app/public'), $request->input('file'));
         return response()->download($path);
@@ -16,8 +22,7 @@

-    // VULNERABILITY: Path Traversal — read arbitrary file
     public function previewImage(Request $request)
     {
-        $file = $request->input('path');
-        $content = file_get_contents(base_path($file)); // No path validation
-        return response($content)->header('Content-Type', 'image/jpeg');
+        $path = $this->safePath(storage_path('app/public/images'), $request->input('path'));
+        abort_unless(in_array(mime_content_type($path), ['image/jpeg', 'image/png'], true), 415);
+        return response()->file($path);
     }
```

`app/Models/Post.php` dan `app/Models/User.php`

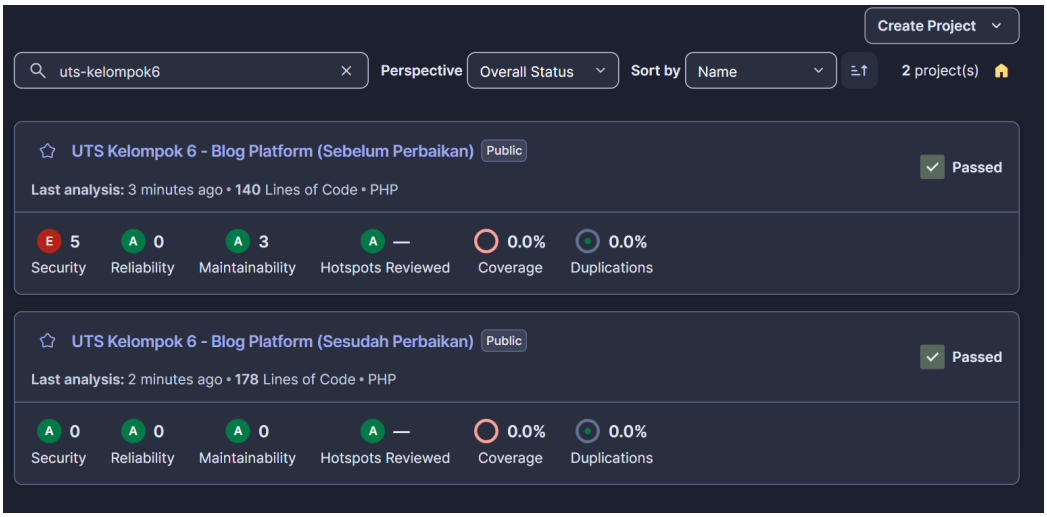
```
@@ -6,4 +6,4 @@
 {
-    // VULNERABILITY: Mass Assignment — $fillable not defined, $guarded empty
-    protected $guarded = [];
+    // FIX: Mass Assignment, hanya kolom berikut yang boleh diisi massal
+    protected $fillable = ['title', 'content', 'category', 'user_id'];
```

```
@@ -6,4 +6,5 @@
 {
-    // VULNERABILITY: Mass Assignment — fillable not restricted
-    protected $fillable = [];
+    // FIX: Mass Assignment, kolom sensitif (mis. is_admin) tidak bisa diisi massal
+    protected $fillable = ['name', 'email', 'password'];
+    protected $hidden = ['password', 'remember_token'];
```

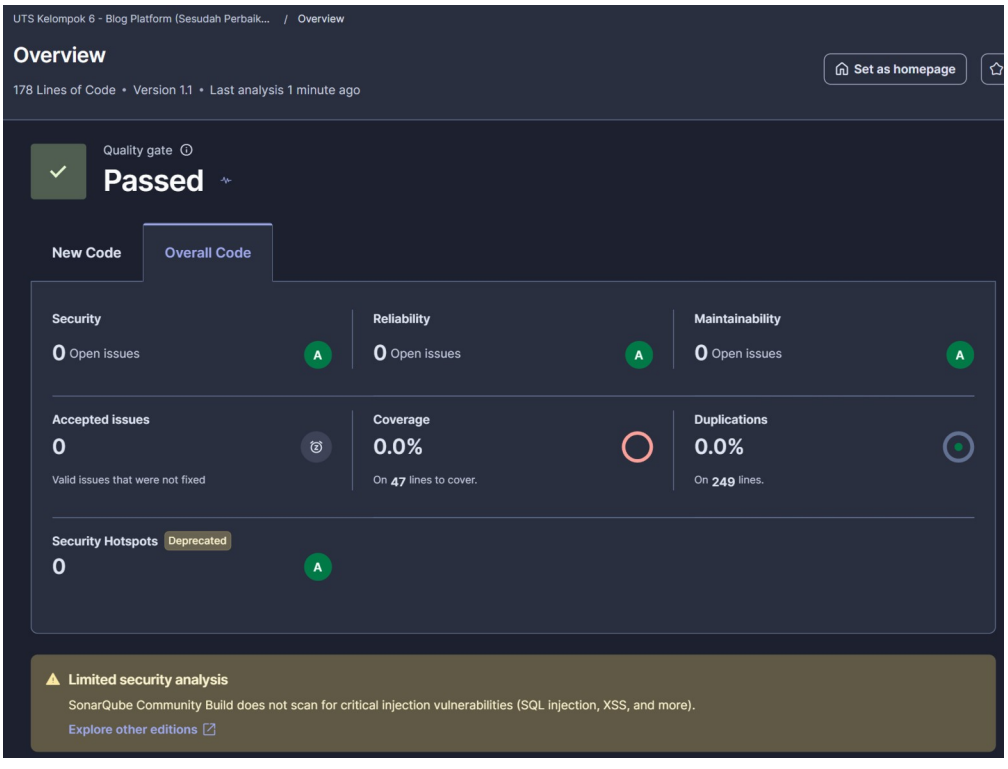
Karena SonarQube Community Build tidak mendeteksi kerentanan injeksi, perbaikan pada 4.5 tidak dapat dibuktikan melalui perubahan jumlah issue. Bukti untuk bagian ini adalah diff kode di atas; pengujian dinamis (DAST) direkomendasikan sebagai verifikasi lanjutan.

4.6 Verifikasi Re-scan

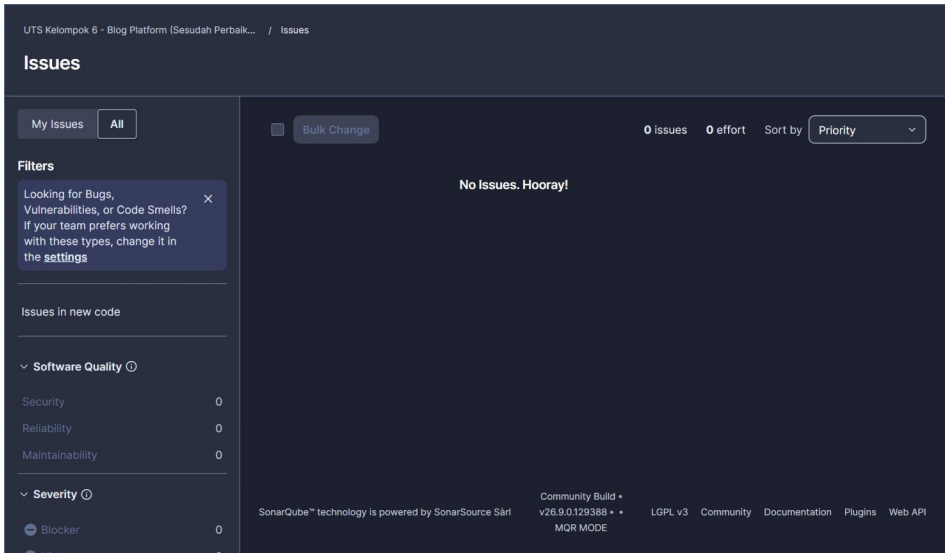
Kode hasil perbaikan di-scan ulang ke project **uts-kelompok6-blog-sesudah** dengan konfigurasi scanner yang sama (sources `app,routes,config`). Hasilnya: **0 issue** di semua kategori dan Security Rating naik dari **E menjadi A**. Kedua project ditampilkan berdampingan pada Gambar 4.5.



Gambar 4.5 *Perbandingan project sebelum (Security E, 5 issue) dan sesudah perbaikan (Security A, 0 issue)*



Gambar 4.6 *Dashboard project sesudah perbaikan (Overall Code)*



Gambar 4.7 *Halaman Issues project sesudah perbaikan: 0 issues*

Tabel 4.2 *Perbandingan metrik SonarQube sebelum dan sesudah perbaikan*

Metrik	Sebelum	Sesudah
Security issues	5	0
Security Rating	**E**	**A**
Vulnerability (klasifikasi lama)	4	0
Reliability issues / Rating	0 / A	0 / A
Maintainability issues / Rating	3 / A	0 / A
Security Hotspots	0	0
Lines of Code	140	178
Quality Gate	Passed	Passed

Tabel 4.3 *Status setiap temuan SonarQube setelah re-scan*

Rule	Lokasi Awal	Severity	Status
secrets:S6334	AdminController.php : 9	**Blocker**	Teratasi
php:S4790	AuthController.php : 12	**High**	Teratasi
php:S4790	AuthController.php : 25	**High**	Teratasi
php:S4502	VerifyCsrfToken.php : 12	**High**	Teratasi
php:S1313	config/database.php : 15	**Low**	Teratasi
php:S1068	AdminController.php : 9	**Medium**	Teratasi
php:S1068	AdminController.php : 10	**Medium**	Teratasi
php:S1135	VerifyCsrfToken.php : 10	**Info**	Teratasi

Jumlah baris kode bertambah dari 140 menjadi 178 karena penambahan validasi input dan file `config/services.php`. Coverage tetap 0% karena project ini tidak

memiliki unit test; hal ini tidak memengaruhi Quality Gate pada analisis pertama, tetapi akan menjadi syarat ketika project berlanjut (lihat rekomendasi di Bab V).

BAB V KESIMPULAN

5.1 Kesimpulan

Audit SAST dengan SonarQube terhadap STT-NF Blog Platform menemukan **8 issue**: 5 Security (1 Blocker, 3 High, 1 Low) dan 3 Maintainability, dengan Security Rating E. Kerentanan utamanya adalah Google API key dan password yang ditulis di kode, password pengguna yang di-hash dengan MD5, serta proteksi CSRF yang dimatikan untuk seluruh route. Setelah semua temuan diperbaiki, re-scan pada project terpisah menunjukkan **0 issue** dan Security Rating A.

Review manual menemukan kerentanan yang lebih berbahaya tetapi tidak dilaporkan SonarQube Community Build, yaitu SQL Injection, OS Command Injection, Path Traversal, IDOR, dan Mass Assignment. Semuanya juga sudah diperbaiki di kode.

5.2 Pelajaran yang Diambil

- **Hasil bersih belum tentu aman.** Dashboard dengan 0 issue dan Quality Gate Passed hanya berarti tidak ada pelanggaran terhadap rule yang aktif. Pada audit ini Quality Gate sudah Passed bahkan sebelum perbaikan, karena analisis pertama tidak memiliki "new code".
- **Kenali batas tool.** Edisi Community tidak memiliki taint analysis, sehingga celah injeksi yang paling kritis justru tidak terlihat. Review manual tetap wajib.
- **Secret yang pernah di-commit harus dianggap bocor.** Menghapus baris kode tidak menghapus riwayat git; secret harus dirotasi.
- **Gunakan fitur keamanan bawaan framework.** Laravel sudah menyediakan bcrypt (`Hash`), CSRF middleware, parameter binding, validasi, dan `$fillable`. Hampir semua kerentanan muncul karena fitur tersebut dimatikan atau dilewati.
- **Konfigurasi tool juga perlu diaudit.** `sonar-project.properties` bawaan menyimpan kredensial dan tidak memindai folder `config/`, sehingga hardcoded IP di file konfigurasi tidak akan terdeteksi jika pengaturan bawaan dipakai apa adanya.

5.3 Rekomendasi Keamanan untuk Project Sejenis

1. Jalankan SonarQube otomatis di pipeline CI/CD (misalnya GitHub Actions) pada setiap push dan pull request, dan jadikan Quality Gate sebagai syarat merge.
2. Lengkapi SAST dengan taint analysis (SonarQube Developer Edition atau tool lain seperti Semgrep), DAST (OWASP ZAP) pada environment staging, serta SCA (`composer audit`) untuk dependensi.
3. Simpan seluruh secret di `.env` atau secret manager, tambahkan secret scanning sebelum commit (misalnya gitleaks sebagai pre-commit hook), dan aktifkan push protection di GitHub.

4. Tetapkan standar coding aman Laravel untuk tim: selalu memakai Eloquent atau parameter binding, `$request->validate()`, `$fillable`, Policy/Gate untuk otorisasi kepemilikan data, dan larang `exec`/`shell_exec` dengan input pengguna.
5. Tambahkan unit dan feature test, terutama untuk autentikasi dan otorisasi, agar syarat coverage pada Quality Gate dapat terpenuhi dan regresi keamanan cepat terdeteksi.
6. Lakukan rotasi semua kredensial yang pernah tertulis di kode (API key Google, password SMTP, password database root dan backup) dan paksa reset password pengguna yang masih memakai hash MD5.

LAMPIRAN

A. Repositori dan Riwayat Commit

Source code sebelum dan sesudah perbaikan tersimpan di repositori GitHub kelompok:

`https://github.com/FirmansyahDzakwanArifien/DevSecOPS/tree/uts/sast-sonarqube-kelompok6/UTS_Kelompok-6_SAST`

Tabel L.1 *Riwayat commit perbaikan*

No	Commit	Isi Perubahan
1	chore: tambah source kelompok-A-blog-platform	Source code asli sebelum audit (baseline).
2	fix: pindahkan API key, password SMTP & kredensial DB ke environment	secrets:S6334, php:S1068, php:S1313, command injection.
3	fix: ganti hash MD5 dengan bcrypt dan parameter binding pada login	php:S4790, SQL Injection login.
4	fix: aktifkan kembali proteksi CSRF untuk semua route	php:S4502, php:S1135.
5	fix: perbaiki SQL injection, IDOR, mass assignment dan path traversal	Temuan review manual.
6	chore: hapus kredensial dari sonar-project.properties	Konfigurasi scanner.

B. Project SonarQube

Sebelum perbaikan	https://sonarqube.cyphera.my.id/dashboard?id=uts-kelompok6-blog-sebelum
Sesudah perbaikan	https://sonarqube.cyphera.my.id/dashboard?id=uts-kelompok6-blog-sesudah

C. Diff Konfigurasi Scanner

`sonar-project.properties`

```
@@ -2,8 +2,7 @@
 sonar.projectName=STT-NF Blog Platform
-sonar.projectVersion=1.0
-sonar.sources=app,routes
+sonar.projectVersion=1.1
+sonar.sources=app,routes,config
 sonar.exclusions=vendor/**,storage/**,bootstrap/cache/**
 sonar.php.version=8.1
-sonar.host.url=http://localhost:9000
-sonar.login=admin
-sonar.password=admin
+# Host & token diberikan lewat environment variable SONAR_HOST_URL dan SONAR_TOKEN,
+# tidak ditulis di file (sonar.login/sonar.password sudah tidak didukung SonarQube terbaru)
```